PRACTICAL NUMBER: 4

AIM: Program based on Hashing (Separate Chaining method)

***PROGRAM*:**

```
//sheikh wasimuddin cse(iot) 60  23071044 //
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
   int key;
   struct Node* next;
};
struct HashTable {
   int size;
   struct Node** table;
};
struct Node* createNode(int key) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
   newNode->key = key;
   newNode->next = NULL;
   return newNode;
}
struct HashTable* createTable(int size) {
```

```
    int i;
        struct HashTable* newTable = (struct
HashTable*)malloc(sizeof(struct HashTable));
    newTable->size = size;
    newTable->table = (struct Node**)malloc(size * sizeof(struct
Node));
    for (i = 0; i < size; i++) {
        newTable->table[i] = NULL;
    }
    return newTable;
}
int hashFunction(int key, int size) {
    return key % size;
}
void insert(struct HashTable* hashTable, int key) {
    int index = hashFunction(key, hashTable->size);
    struct Node* newNode = createNode(key);
    newNode->next = hashTable->table[index];
    hashTable->table[index] = newNode;
}
void display(struct HashTable* hashTable) {
    int i;
        for ( i = 0; i < hashTable->size; i++) {
        struct Node* temp = hashTable->table[i];
```

```
        printf("Chain %d: ", i);
        while (temp != NULL) {
            printf("%d -> ", temp->key);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}
int main() {
    int size = 10;
    struct HashTable* hashTable = createTable(size);

    insert(hashTable, 43);
    insert(hashTable, 76);
    insert(hashTable, 75);
    insert(hashTable, 19);
    insert(hashTable, 77);
    insert(hashTable, 12);
    insert(hashTable, 53);
    insert(hashTable, 73);
    display(hashTable);
    return 0;
}
```

OUTPUT:

main.c

```c
//sheikh wasimuddin cse(iot) 60  23071044 //

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
    int key;
    struct Node* next;
};
struct HashTable {
    int size;
    struct Node** table;
};
struct Node* createNode(int key) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->key = key;
    newNode->next = NULL;
    return newNode;
}
struct HashTable* createTable(int size) {
    int i;
    struct HashTable* newTable = (struct HashTable*)malloc(sizeof(struct HashTable));
    newTable->size = size;
    newTable->table = (struct Node**)malloc(size * sizeof(struct Node));
    for (i = 0; i < size; i++) {
        newTable->table[i] = NULL;
    }
    return newTable;
}
int hashFunction(int key, int size) {
    return key % size;
}
void insert(struct HashTable* hashTable, int key) {
    int index = hashFunction(key, hashTable->size);
    struct Node* newNode = createNode(key);
    newNode->next = hashTable->table[index];
```

```
Chain 0: NULL
Chain 1: NULL
Chain 2: 12 -> NULL
Chain 3: 73 -> 53 -> 43 -> NULL
Chain 4: NULL
Chain 5: 75 -> NULL
Chain 6: 76 -> NULL
Chain 7: 77 -> NULL
Chain 8: NULL
Chain 9: 19 -> NULL


...Program finished with exit code 0
Press ENTER to exit console.
```

CONCLUSION: Program based on Hashing (Separate Chaining method)is successfully performed.